TRƯỜNG ĐẠI HỌC ĐỒNG THÁP
Khoa Công nghệ và Kỹ thuật

`IN4211 · CHƯƠNG 3`

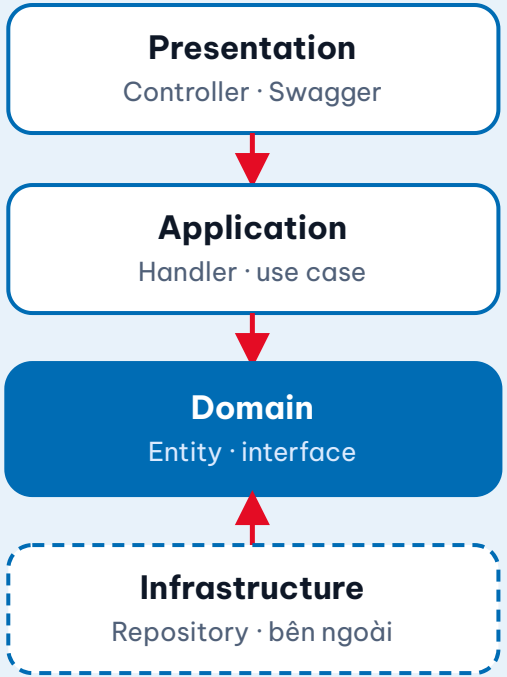
Clean Architecture và REST API cơ bản

Phát triển phần mềm hướng đối tượng

Giảng viên **ThS. Nguyễn Tuấn Kiệt**
Năm học 2026–2027

`.NET 10` `API` `Swagger` `Moq`

0836 980 284 nguyentuankiet@dthu.edu.vn



Domain: chỉ khai báo IRepository
Infrastructure: viết code lưu thật

Domain không có dòng using nào trỏ ra ngoài

Nội dung chương 3

🏛️ **3.1** — Kiến trúc 4 lớp: Domain, Application, Infrastructure, Presentation

🔌 **3.2** — Dependency Inversion và DI container

🎮 **3.3** — Controller-based API, Minimal API để so sánh, thiết kế route

📄 **3.4** — Tài liệu API với Swagger UI, HTTP status code chuẩn

🧪 **3.5** — Unit test lớp Application, mock bằng Moq

🛠️ **Bài tập Ch03** — EduHub Skeleton

Học xong chương này bạn làm được gì?

CLO1	Dựng khung Clean Architecture 4 lớp, giải thích được chiều phụ thuộc giữa các lớp
CLO2	Viết REST API bằng ASP.NET Core đúng route và status code, có Swagger; quản lý code bằng Git và GitHub
CLO4	Áp dụng vào EduHub: dựng bộ khung ứng dụng quản lý câu lạc bộ
CLO5	Trình bày kết quả kỹ thuật trước lớp và nhận phản biện mang tính xây dựng
CLO7	Tự nghiên cứu từ tài liệu chính thức để hiểu công nghệ mới

Cách học chương 3: nghiên cứu, thuyết trình, phản biện



Hình 3.1. Quy trình mỗi nhóm thuyết trình (nhóm 2–3 sinh viên) (Nguồn: đề cương học phần, mục 10)

Nộp slide trễ: không có phản hồi trước và bị trừ 30% điểm thuyết trình. Không có commit chỉnh sửa sau phản hồi: mất toàn bộ 10% điểm phần phản hồi.

Tài liệu nghiên cứu cho chương 3

TÀI LIỆU	TÁC GIẢ	DÙNG CHO MỤC
[1] *.NET 10 Documentation* — Microsoft Learn learn.microsoft.com/dotnet	Microsoft Corporation	3.2 đến 3.5
[2] *Clean Architecture: A Craftsman's Guide to Software Structure and Design*	Robert C. Martin, 2017	3.1, 3.2
[3] *ASP.NET Core in Action*, 2nd ed.	Andrew Lock, 2021	3.3, 3.4

Bảng 3.1. Tài liệu chính của chương 3 (theo đề cương)

Khi chuẩn bị slide, ưu tiên tài liệu chính thức và ghi nguồn cho từng hình, từng đoạn trích.

Ôn nhanh từ Chương 2

Q1 **DIP (đảo ngược phụ thuộc) là gì?**

Code nghiệp vụ không gọi thẳng code chi tiết như SQL hay gửi email. Cả hai cùng dựa vào một **interface** (hợp đồng).

Q2 **Repository dùng để làm gì?**

Gom việc lấy và lưu dữ liệu vào một chỗ. Code nghiệp vụ chỉ biết interface, không biết SQL.

Q3 **Ba bước trong một unit test (AAA) là gì?**

Arrange (chuẩn bị), Act (thực hiện), Assert (kiểm tra kết quả).

Q4 **Commit thêm tính năng mới dùng tiền tố nào?**

`feat:` — ví dụ `feat(club): add create club endpoint`

1

PHẦN 1 / 5

PHẦN 1 · MỤC 3.1

Kiến trúc 4 lớp

- “God Method” và vì sao cần kiến trúc
- Clean Architecture: lớp lõi và Dependency Rule
- Domain, Application, Infrastructure, Presentation
- Khung 5 project của EduHub
- Một request đi qua các lớp

“God Method” – một hàm làm tất cả



Hình 3.2. Một hàm ôm đồm mọi việc (Nguồn: tự vẽ)

Giống quán nhỏ: một người vừa nấu, vừa thu tiền, vừa rửa chén. **Rất dễ sai.**

God Method — nhìn vào code

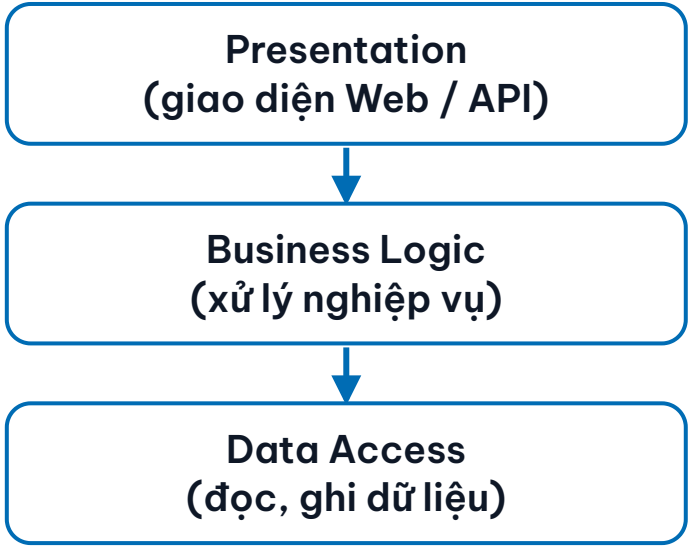
```
[HttpPost]
public IActionResult CreateClub(string name, string description)
{
    // 1. Kiểm tra dữ liệu ngay trong controller
    if (string.IsNullOrWhiteSpace(name)) return BadRequest("Tên rỗng");

    // 2. Gọi thẳng database, ghép chuỗi SQL: kẻ xấu chèn được lệnh (SQL Injection)
    var conn = new SqlConnection("Server=.;Database=EduHub");
    var sql = $"INSERT INTO Clubs VALUES ('{name}', '{description}')";
    new SqlCommand(sql, conn).ExecuteNonQuery();

    // 3. Gửi email thật ngay trong hàm: chạy test cũng gửi mail
    new SmtpClient("smtp.eduhub.vn").Send("admin@eduhub.vn", "New club");
    return Content("<h1>Tạo thành công!</h1>");   // 4. Tự ghép HTML trả về
}
```

Đoạn mã 3.1. Controller ôm đồm kiểm tra, truy cập database và gửi email

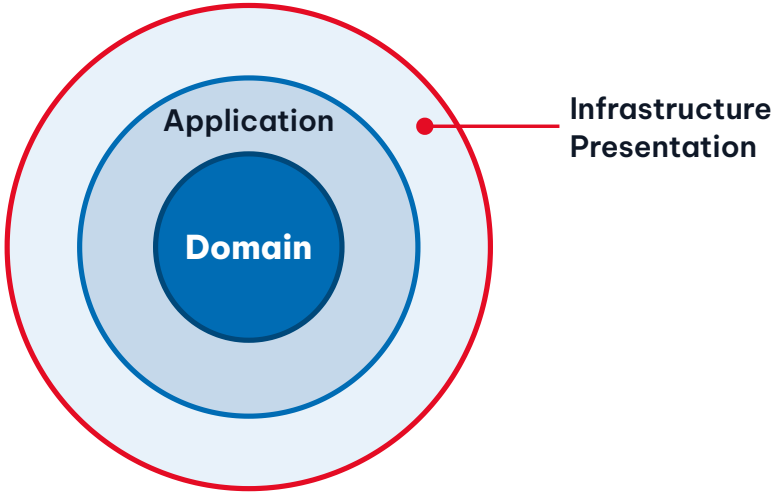
1.3 Phân tầng 3 lớp tốt hơn, nhưng chưa đủ



Tốt hơn God Method. Giao diện, luật nghiệp vụ (ví dụ: tên CLB không rỗng) và dữ liệu nằm ở các chỗ riêng.

Điểm yếu còn lại: tầng logic gọi thẳng tầng dữ liệu nên dính chặt vào nó. Đổi cách lưu dữ liệu, hay test mà không cần database thật, đều khó.

Hình 3.3. Kiến trúc 3 tầng truyền thống (Nguồn: tự vẽ)



Dependency Rule (luật phụ thuộc):
lớp ngoài được biết lớp trong.
Lớp trong **không** biết lớp ngoài.

Mũi tên "A → B" nghĩa là A dùng B:
Presentation → Application → Domain
Infrastructure → Application → Domain

Hình 3.4. Các lớp đồng tâm, phụ thuộc chỉ hướng vào lõi
(Nguồn: phỏng theo R. C. Martin, *Clean Architecture*, 2017)

LỚP	VAI TRÒ	VÍ DỤ TRONG EDUHUB	ĐƯỢC PHÉP BIẾT
Domain	Luật nghiệp vụ cốt lõi	Club, Member, IClubRepository	Không lớp nào
Application	Từng việc người dùng nhờ (use case)	CreateClubCommand, GetClubsQueryHandler	Domain
Infrastructure	Làm việc thật với bên ngoài	InMemoryClubRepository (Chương 4: EF Core)	Application, Domain
Presentation	Nhận request, trả response	ClubsController, Swagger	Application (và Infrastructure để đăng ký DI)

Bảng 3.2. Bốn lớp của EduHub

1.6 Domain — lõi không phụ thuộc ai

✅ Domain chứa

- Entity có Id riêng: Club, Member
- Interface kho dữ liệu: IClubRepository
- Luật nghiệp vụ: tên CLB không rỗng
- Lỗi riêng của bài toán

❌ Domain không chứa

- Thư viện EF Core, ASP.NET Core
- SQL, HTTP, gửi email
- Tham chiếu tới lớp ngoài
- Biết dữ liệu lưu ở đâu

Thử kiểm tra: copy project Domain sang dự án khác, nó vẫn build được mà không cần thêm gì. Đó là dấu hiệu Domain đúng.

Domain – entity Club

```
public class Club
{
    public Guid Id { get; private set; } = Guid.NewGuid();
    public string Name { get; private set; } = string.Empty;
    public string Description { get; private set; } = string.Empty;
    public DateTime CreatedAt { get; private set; } = DateTime.UtcNow;

    private Club() { }

    // Luật của bài toán: tạo CLB bằng Club.Create(), tên không được rỗng
    public static Club Create(string name, string description)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(name);
        return new Club { Name = name, Description = description };
    }
}
```

Đoạn mã 3.2. Entity Club. private set: chỉ sửa được từ bên trong class; Guid: mã định danh không trùng

1.8 Domain — interface IClubRepository

```
namespace EduHub.Domain.Interfaces;

public interface IClubRepository
{
    Task<IEnumerable<Club>> GetAllAsync(CancellationToken ct = default);
    Task<Club?> GetByIdAsync(Guid id, CancellationToken ct = default);
    Task AddAsync(Club club, CancellationToken ct = default);
    Task UpdateAsync(Club club, CancellationToken ct = default);
    Task DeleteAsync(Guid id, CancellationToken ct = default);
}
```

Đoạn mã 3.3. Hợp đồng của kho CLB: Domain khai báo, Infrastructure cài đặt

Task: việc chạy bất đồng bộ (async). Club?: có thể là null khi không tìm thấy. ct (CancellationToken): tín hiệu để huỷ việc đang chạy.

1.9 Application — mỗi use case một handler

✏️ Command (lệnh): ghi

Làm thay đổi dữ liệu

- CreateClubCommand
- DeleteClubCommand
- AddMemberCommand (Chương 4)

👀 Query (câu hỏi): đọc

Chỉ xem, không đổi gì

- GetClubsQuery
- GetClubByIdQuery
- GetMembersQuery (Chương 4)

Handler là class xử lý đúng một lệnh hoặc một câu hỏi. Handler chỉ biết interface của Domain, không biết dữ liệu nằm trong bộ nhớ hay database.

Application — Command và handler

```
public record CreateClubCommand(string Name, string Description);
public class CreateClubCommandHandler
{
    private readonly IClubRepository _repository;   // chỉ biết interface

    public CreateClubCommandHandler(IClubRepository repository) => _repository = repository;

    public async Task<Club> HandleAsync(
        CreateClubCommand command, CancellationToken ct = default)
    {
        // 1. Kiểm tra đầu vào: tên không rỗng, tối đa 50 ký tự (xem mục 3.4)
        // 2. Tạo entity bằng factory method của Domain
        var club = Club.Create(command.Name.Trim(), command.Description);
        await _repository.AddAsync(club, ct);       // 3. Nhờ kho lưu lại
        return club;
    }
}
```

Đoạn mã 3.4. Handler tạo CLB: kiểm tra, tạo entity, nhờ kho lưu lại

Infrastructure và Presentation

🧰 Infrastructure

Làm việc thật với bên ngoài

- InMemoryClubRepository: lưu trong bộ nhớ (Chương 3)
- Bản dùng EF Core và PostgreSQL (Chương 4)
- Dịch vụ ngoài: email, token (các chương sau)

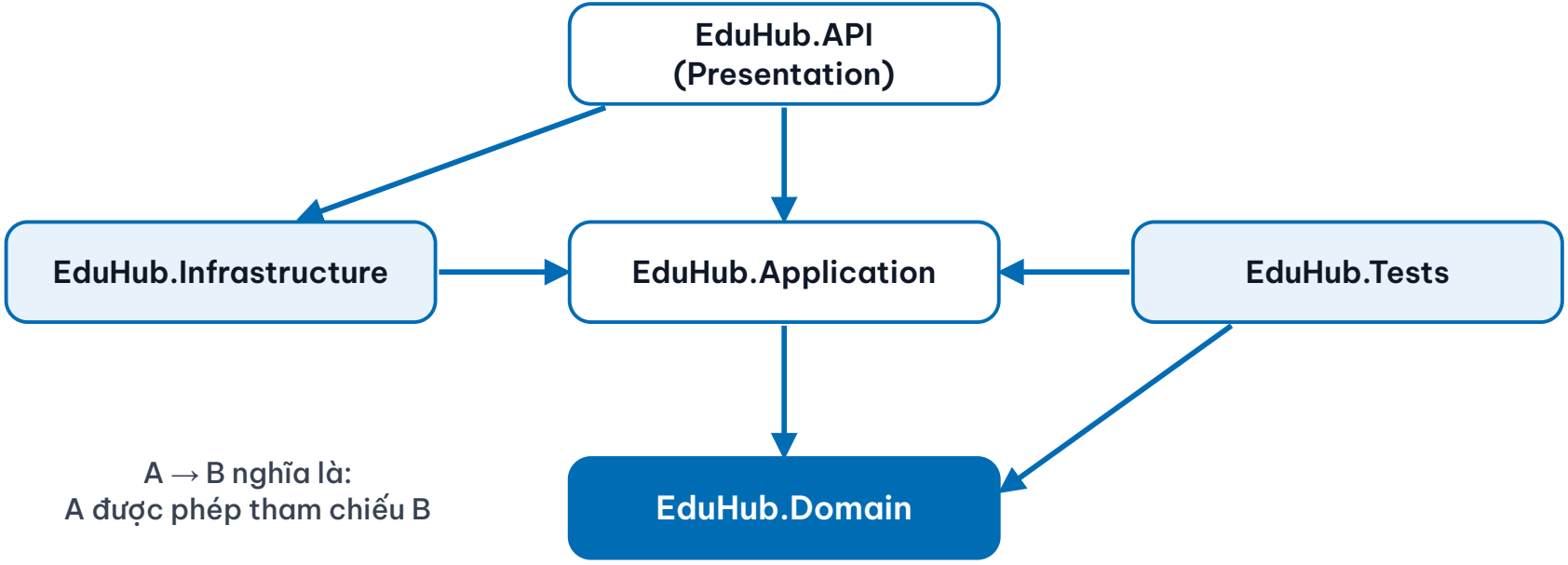
🎮 Presentation (EduHub.API)

Cửa ngõ của hệ thống

- Controller: nhận request, trả response
- Swagger: tài liệu và trang thử API
- Program.cs: đăng ký DI, dựng pipeline

Đổi từ bộ nhớ sang database (Chương 4) chỉ phải sửa Infrastructure và một dòng đăng ký DI. Domain và Application giữ nguyên.

Khung giải pháp: 5 project



Hình 3.5. Chiều tham chiếu giữa 5 project của EduHub (Nguồn: tự vẽ)

```
dotnet new sln -n EduHub --format sln
dotnet new classlib -n EduHub.Domain
dotnet new classlib -n EduHub.Application
dotnet new classlib -n EduHub.Infrastructure
dotnet new webapi --use-controllers -n EduHub.API
dotnet new xunit -n EduHub.Tests

dotnet sln add EduHub.Domain EduHub.Application EduHub.Infrastructure \
    EduHub.API EduHub.Tests
```

Đoạn mã 3.5. Dựng solution gồm 5 project

sln: solution, cái "thùng" chứa các project. classlib: project chỉ chứa code, không tự chạy.

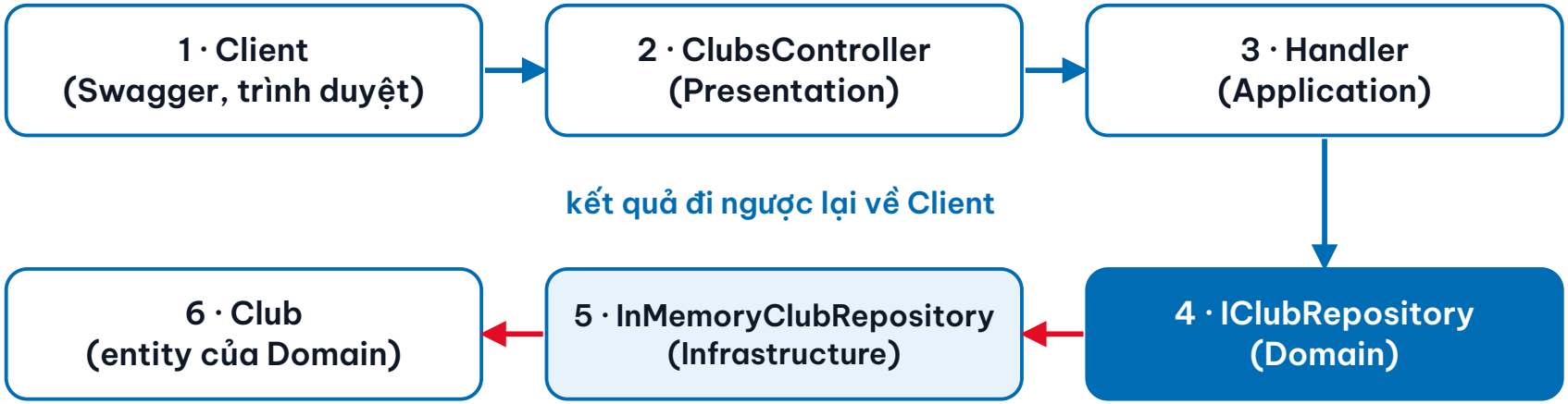
Thêm tham chiếu và 3 điều cấm

```
dotnet add EduHub.Application reference EduHub.Domain
dotnet add EduHub.Infrastructure reference EduHub.Application
dotnet add EduHub.API reference EduHub.Application EduHub.Infrastructure
dotnet add EduHub.Tests reference EduHub.Application EduHub.Domain
```

Đoạn mã 3.6. Nối các project theo chiều phụ thuộc

Ba tham chiếu bị cấm: Domain → Application, Domain → Infrastructure, Application → Infrastructure. Lớp trong mà dùng lớp ngoài thì đổi database là phải sửa cả Domain.

Một request đi qua các lớp



Hình 3.6. Luồng GET /api/clubs trong EduHub (Nguồn: tự vẽ)

Handler gọi interface (bước 4); DI container đưa bản InMemory (bước 5) vào lúc chạy.

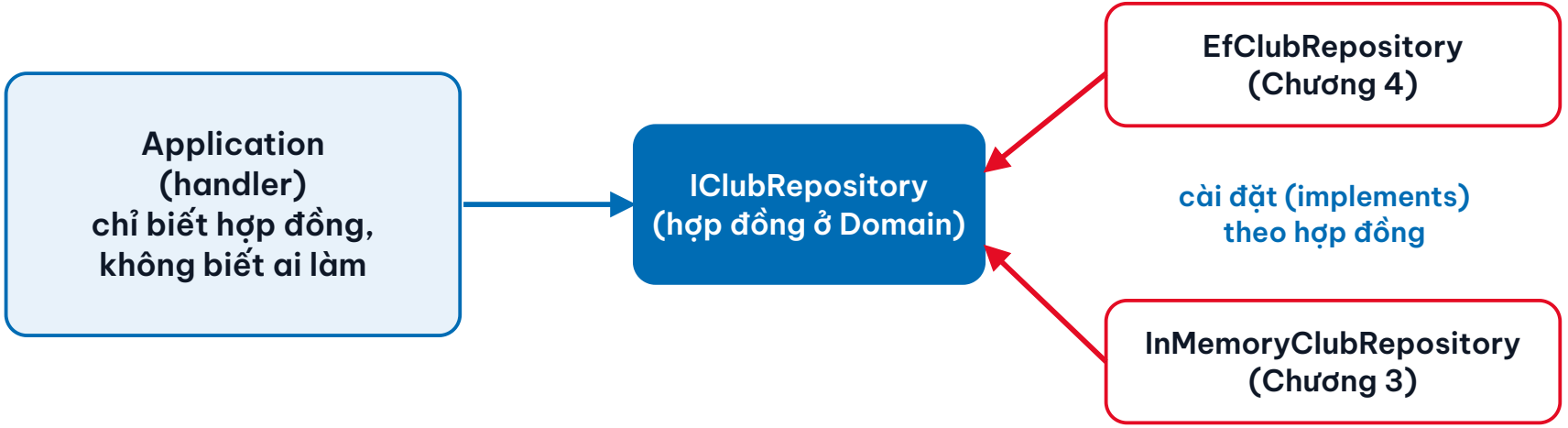
2

PHẦN 2 / 5

PHẦN 2 · MỤC 3.2

Dependency Inversion và DI container

- Interface là “hợp đồng”
- Dependency Inversion trong code
- DI container và đăng ký trong Program.cs
- Ba vòng đời: Transient, Scoped, Singleton



Hình 3.7. Application và Infrastructure cùng dựa vào interface (Nguồn: tự vẽ)

Dependency Inversion: module cấp cao không phụ thuộc module cấp thấp; cả hai cùng phụ thuộc abstraction.

```
// Domain: hợp đồng
public interface IClubRepository { /* GetAllAsync, GetByIdAsync, AddAsync, ... */ }

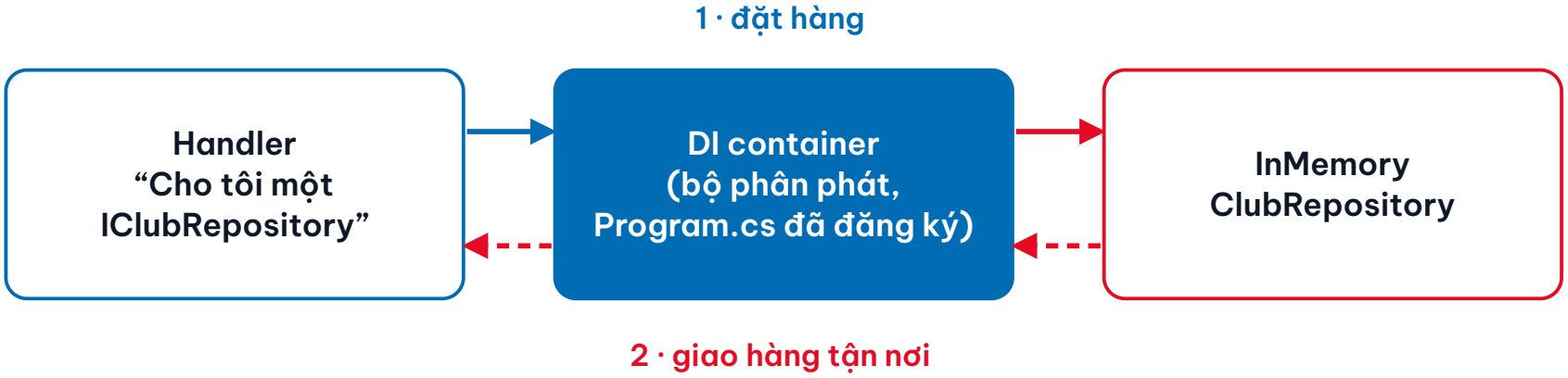
// Infrastructure: làm theo hợp đồng
public class InMemoryClubRepository : IClubRepository { /* ... */ }

// Application: chỉ xin hợp đồng qua constructor
public class GetClubsQueryHandler
{
    private readonly IClubRepository _repository;
    public GetClubsQueryHandler(IClubRepository repository) => _repository = repository;
}
```

Đoạn mã 3.7. Ba lớp, một hợp đồng

Handler không có chữ "InMemory" nào. Bản cụ thể được chọn ở nơi khác (slide sau).

2.3 DI container — đặt hàng thay vì tự đi chợ



Hình 3.8. Class chỉ nói cần gì; container lo đưa cái gì tới (Nguồn: tự vẽ)

DI (Dependency Injection) = class nhận thứ nó cần từ bên ngoài, thay vì tự **new** ra.

Đăng ký DI trong Program.cs

```
// Ai xin interface này thì đưa class kia
builder.Services.AddScoped<IClubRepository, InMemoryClubRepository>();
builder.Services.AddScoped<IMemberRepository, InMemoryMemberRepository>();

// Handler cũng đăng ký để container dựng và đưa vào controller
builder.Services.AddScoped<GetClubsQueryHandler>();
builder.Services.AddScoped<GetClubByIdQueryHandler>();
builder.Services.AddScoped<CreateClubCommandHandler>();
builder.Services.AddScoped<DeleteClubCommandHandler>();

// Chương 4: đổi sang EfClubRepository, chỉ sửa đúng một dòng ở trên
```

Đoạn mã 3.8. Đăng ký DI cho EduHub

Container tự dựng cả chuỗi: Controller cần Handler, Handler cần Repository. Ta chỉ khai báo, không tự new.

2.5 Ba vòng đời (lifetime) của dịch vụ

CÁCH ĐĂNG KÝ	CONTAINER TẠO BẢN MỚI KHI NÀO	HỢP VỚI
AddTransient	Mỗi lần có nơi xin	Dịch vụ nhẹ, không giữ trạng thái
AddScoped	Một bản cho mỗi request HTTP	Handler, repository, DbContext (Chương 4)
AddSingleton	Một bản cho cả ứng dụng	Cấu hình, bộ nhớ đệm

Bảng 3.3. Ba vòng đời của DI trong ASP.NET Core

EduHub dùng **Scoped** cho handler và repository: hết request thì bỏ, request sau nhận bản mới.

2.6 Câu hỏi hay: Scoped nhưng dữ liệu vẫn còn?

❌ Danh sách thường

```
private readonly List<Club> _clubs = new();
```

Mỗi request có repository mới nên có danh sách rỗng mới. POST xong, GET vẫn thấy rỗng.

✅ Danh sách static

```
private static readonly List<Club> _clubs = new();
private static readonly object _lock = new();
```

Dùng chung giữa các request; thêm lock để nhiều request cùng lúc không làm hỏng danh sách.

> Đây chỉ là giải pháp tạm. Sang Chương 4, dữ liệu nằm trong PostgreSQL nên không còn vấn đề này.

3

PHẦN 3 / 5

PHẦN 3 · MỤC 3.3

Controller-based API, Minimal API, route

- REST và các động từ HTTP
- Thiết kế route
- ClubsController của EduHub
- Minimal API để so sánh

PHƯƠNG THỨC	Ý NGHĨA	AN TOÀN	IDEMPOTENT
GET	Đọc dữ liệu	Có	Có
POST	Tạo mới	Không	Không
PUT	Thay toàn bộ một tài nguyên	Không	Có
PATCH	Sửa một phần	Không	Không
DELETE	Xoá	Không	Có

Bảng 3.4. Các phương thức HTTP thường dùng

An toàn: không làm đổi dữ liệu. **Idempotent**: gọi nhiều lần cho kết quả như gọi một lần (xoá hai lần vẫn là "đã không còn").

3.2 Thiết kế route cho EduHub

VIỆC CẦN LÀM	ROUTE NÊN TRÁNH	ROUTE NÊN DÙNG
Tạo CLB	POST /api/createClub	**POST** /api/clubs
Lấy danh sách	GET /api/getAllClubs	**GET** /api/clubs
Xoá một CLB	POST /api/club/delete?id=7	**DELETE** /api/clubs/{id}

Bảng 3.5. Đặt tên route theo tài nguyên, để động từ HTTP nói lên hành động

- Danh từ số nhiều cho tài nguyên (clubs, members); không đặt động từ trong URL
- Id nằm trong đường dẫn; quan hệ cha con viết lồng nhau: /api/clubs/{id}/members
- Lọc, tìm kiếm dùng query string: /api/clubs?name=lap-trinh

ClubsController — nhận request, gọi handler (1)

```
[ApiController]
[Route("api/clubs")]
public class ClubsController : ControllerBase
{
    [HttpGet]                                   // GET api/clubs → 200
    public async Task<IActionResult> GetAll(CancellationToken ct)
    {
        var clubs = await _getClubsHandler.HandleAsync(new GetClubsQuery(), ct);
        return Ok(clubs);
    }

    [HttpGet("{id:guid}")]                      // GET api/clubs/{id} → 200 hoặc 404
    public async Task<IActionResult> GetById(Guid id, CancellationToken ct)
    {
        var club = await _getClubByIdHandler.HandleAsync(new GetClubByIdQuery(id), ct);
        return club is null ? NotFound() : Ok(club);
    }
```

Đoạn mã 3.9. Controller: hai endpoint đọc dữ liệu (các handler nhận qua constructor nhờ DI)

```
[HttpPost]                                          // POST api/clubs → 201 hoặc 400
public async Task<IActionResult> Create(
    [FromBody] CreateClubRequest request, CancellationToken ct)
{
    try
    {
        var command = new CreateClubCommand(request.Name, request.Description);
        var club = await _createClubHandler.HandleAsync(command, ct);
        return CreatedAtAction(nameof(GetById), new { id = club.Id }, club);
    }
    catch (ArgumentException ex)
    {
        return BadRequest(new { message = ex.Message });    // dữ liệu sai → 400
    }
}
```

Đoạn mã 3.10. Controller: endpoint tạo CLB, trả 201 kèm Location hoặc 400

ClubsController — nhận request, gọi handler (3)

```
    [HttpDelete("{id:guid}")]                    // DELETE api/clubs/{id} → 204 hoặc 404
    public async Task<IActionResult> Delete(Guid id, CancellationToken ct)
    {
        try
        {
            await _deleteClubHandler.HandleAsync(new DeleteClubCommand(id), ct);
            return NoContent();
        }
        catch (KeyNotFoundException) { return NotFound(); }
    }
}
public record CreateClubRequest(string Name, string Description);
```

Đoạn mã 3.11. Controller: endpoint xoá và model nhận dữ liệu tạo CLB

> Controller đang trả thẳng entity Club ra ngoài. Chương 4 sẽ tách DTO và mapping để không lộ cấu trúc nội bộ.

3.6 Các attribute và kết quả trả về

THÀNH PHẦN	TÁC DỤNG
[ApiController]	Bật các quy ước cho API: tự đọc JSON, tự trả 400 khi dữ liệu gửi lên sai kiểu
[Route("api/clubs")]	Địa chỉ gốc của controller
[HttpGet("{id:guid}")]	Nhận GET; ràng buộc id phải là Guid, sai kiểu thì không khớp route
[FromBody]	Đọc JSON trong thân request thành object
Ok · NotFound · CreatedAtAction · NoContent	Hàm dựng kết quả: 200 · 404 · 201 kèm Location · 204

Bảng 3.6. Các thành phần chính của một controller

Controller "mỏng": nhận request, gọi handler, chọn status code. Luật nghiệp vụ nằm ở Domain và Application. Program.cs cần AddControllers() và MapControllers().

Minimal API — giới thiệu ngắn

```
var clubs = app.MapGroup("/api/clubs");

clubs.MapGet("/", async (GetClubsQueryHandler handler, CancellationToken ct) =>
    Results.Ok(await handler.HandleAsync(new GetClubsQuery(), ct)));

clubs.MapGet("/{id:guid}", async (Guid id, GetClubByIdQueryHandler handler,
                                  CancellationToken ct) =>
    await handler.HandleAsync(new GetClubByIdQuery(id), ct) is { } club
        ? Results.Ok(club)
        : Results.NotFound());
```

Đoạn mã 3.12. Hai endpoint đọc, viết bằng Minimal API (không cần class controller)

Cùng handler ở Application, cùng DI, cùng Swagger.
Chỉ khác cách khai báo endpoint ở lớp Presentation.

Controller hay Minimal API?

TIÊU CHÍ	CONTROLLER-BASED	MINIMAL API
Cách viết	Class, method, attribute	Hàm lambda đăng ký trong Program.cs
Tổ chức khi API lớn	Mỗi nhóm endpoint một class, dễ chia file và chia việc	Phải tự chia nhóm (MapGroup, file mở rộng)
Hợp với	API nhiều nhóm tài nguyên, nhiều người cùng làm	API nhỏ, microservice, thử nghiệm nhanh
Với EduHub	**Chọn**: nhiều nhóm (Clubs, Members, Events…)	Biết để đọc code của dự án khác

Bảng 3.7. So sánh hai cách xây API trong ASP.NET Core

4

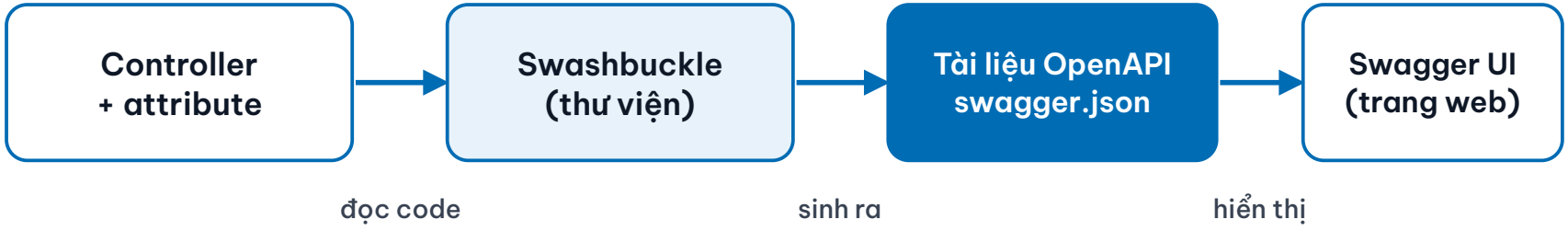
PHẦN 4 / 5

PHẦN 4 · MỤC 3.4

Tài liệu API và HTTP status code

- OpenAPI, Swagger UI, Swashbuckle
- Bật Swagger cho EduHub
- Dùng Swagger UI để thử API
- HTTP status code chuẩn
- Kiểm tra đầu vào ở mức cơ bản

OpenAPI, Swagger UI, Swashbuckle là gì?



Hình 3.9. Từ code đến trang thử API (Nguồn: tự vẽ)

- **OpenAPI**: chuẩn mô tả API bằng JSON, máy đọc được
- **Swagger UI**: trang web đọc tài liệu OpenAPI để xem và gọi thử endpoint
- **Swashbuckle**: thư viện .NET tự sinh tài liệu và kèm sẵn Swagger UI

> Từ .NET 9, template Web API không còn kèm Swashbuckle mà dùng thư viện OpenAPI có sẵn của ASP.NET Core. EduHub vẫn dùng Swashbuckle vì có sẵn Swagger UI.

4.2 Bật Swagger trong Program.cs

```
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
    c.SwaggerDoc("v1", new() { Title = "EduHub API", Version = "v1" }));

var app = builder.Build();

if (app.Environment.IsDevelopment())      // chỉ bật khi đang phát triển
{
    app.UseSwagger();                     // sinh /swagger/v1/swagger.json
    app.UseSwaggerUI();                   // trang thử API: /swagger
}
app.MapControllers();
```

Đoạn mã 3.13. Cấu hình Swagger cho EduHub

Chỉ bật ở môi trường Development để tài liệu API không lộ ra ngoài khi chạy thật.

```
/// <summary>Lấy một câu lạc bộ theo Id.</summary>
[HttpGet("{id:guid}")]
[ProducesResponseType(typeof(Club), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<IActionResult> GetById(Guid id, CancellationToken ct)
{
    var club = await _getClubByIdHandler.HandleAsync(new GetClubByIdQuery(id), ct);
    return club is null ? NotFound() : Ok(club);
}
```

Đoạn mã 3.14. Khai báo các kết quả có thể xảy ra của một endpoint

ProducesResponseType không đổi cách chạy của API. Nó cho Swagger (và người dùng API) biết endpoint có thể trả 200 hoặc 404.

4.4 Dùng Swagger UI để thử API



Hình 3.10. Swagger UI của EduHub: 4 endpoint của Clubs
(Nguồn: ảnh chụp ứng dụng EduHub, giao diện Swagger UI)

1. Chạy API, mở /swagger
2. Chọn endpoint, bấm Try it out
3. Execute, đọc status code và JSON

4.5 HTTP status code: ba nhóm cần nhớ

NHÓM	Ý NGHĨA	HAY GẶP
2xx	Thành công	200 OK · 201 Created · 204 No Content
4xx	Lỗi do phía gọi API	400 Bad Request · 401 Unauthorized · 403 Forbidden · 404 Not Found · 409 Conflict
5xx	Lỗi phía máy chủ	500 Internal Server Error

Bảng 3.8. Các nhóm HTTP status code

401 và 403 liên quan đăng nhập và phân quyền, sẽ gặp ở Chương 5. Quy tắc chọn: client gửi sai thì 4xx, server tự lỗi thì 5xx.

4.6 Chọn status code cho EduHub

ENDPOINT	TÌNH HUỐNG	STATUS CODE
GET /api/clubs	Luôn thành công (có thể là mảng rỗng)	**200**
GET /api/clubs/{id}	Có CLB / không có	**200 / 404**
POST /api/clubs	Tạo được, kèm header Location / dữ liệu sai	**201 / 400**
DELETE /api/clubs/{id}	Xoá được (không có thân trả về) / không có CLB	**204 / 404**

Bảng 3.9. Status code cho 4 endpoint của ClubsController

Kiểm tra đầu vào ở mức cơ bản

```
// Application: handler kiểm tra, sai thì throw
if (string.IsNullOrWhiteSpace(command.Name))
    throw new ArgumentException("Tên câu lạc bộ không được để trống.");

// Presentation: controller đổi lỗi thành status code
catch (ArgumentException ex)
{
    return BadRequest(new { message = ex.Message });   // 400
}
```

Đoạn mã 3.15. Kiểm tra tên CLB ở Application, trả 400 ở Controller

Để Chương 4: DTO và validation chuẩn hoá, xử lý lỗi toàn cục bằng ProblemDetails (RFC 9457), logging có cấu trúc với Serilog. Chương này chỉ cần kiểm tra đơn giản và trả đúng status code.

5

PHẦN 5 / 5

PHẦN 5 · MỤC 3.5

Unit test lớp Application với Moq

- Vì sao lớp Application dễ kiểm thử
- Mock bằng Moq: Setup và Verify
- Ba test mẫu
- Chạy test và đặt tên test



Hình 3.11. Thay kho thật bằng mock khi kiểm thử handler (Nguồn: tự vẽ)

Handler chỉ phụ thuộc interface, nên trong test ta đưa vào một "đồ giả" (mock). Không cần database, không cần chạy web, test chạy trong vài mili giây.

Moq: ba thao tác chính

THAO TÁC	Ý NGHĨA	VÍ DỤ
Tạo mock	Tạo đồ giả của một interface	new Mock<IClubRepository>()
Object	Lấy đối tượng giả đưa vào handler	new CreateClubCommandHandler(mock.Object)
Setup	Dặn đồ giả: gọi hàm này thì trả cái này	mock.Setup(r => r.GetByIdAsync(id, default)).ReturnsAsync(club)
Verify	Kiểm tra đồ giả đã được gọi đúng cách	mock.Verify(r => r.AddAsync(It.IsAny<Club>(), default), Times.Once)

Bảng 3.10. Các thao tác chính của Moq

5.3 Test 1 — tạo CLB hợp lệ

```
[Fact]
public async Task CreateClub_WithValidName_ReturnsCreatedClub()
{
    // Arrange: chuẩn bị mock và handler
    var mockRepo = new Mock<IClubRepository>();
    var handler = new CreateClubCommandHandler(mockRepo.Object);
    var command = new CreateClubCommand("Tech Club", "Câu lạc bộ công nghệ");

    // Act: thực hiện
    var result = await handler.HandleAsync(command);

    // Assert: kiểm tra kết quả và việc kho được gọi đúng 1 lần
    Assert.Equal("Tech Club", result.Name);
    mockRepo.Verify(r => r.AddAsync(It.IsAny<Club>(), default), Times.Once);
}
```

Đoạn mã 3.16. Test tạo CLB hợp lệ theo mẫu Arrange, Act, Assert

Test 2 — dữ liệu sai thì không lưu

```
[Fact]
public async Task CreateClub_WithEmptyName_ThrowsException()
{
    var mockRepo = new Mock<IClubRepository>();
    var handler = new CreateClubCommandHandler(mockRepo.Object);
    var command = new CreateClubCommand("", "Description");

    await Assert.ThrowsAsync<ArgumentException>(() => handler.HandleAsync(command));

    // Tên rỗng thì tuyệt đối không được gọi kho để lưu
    mockRepo.Verify(r => r.AddAsync(It.IsAny<Club>(), default), Times.Never);
}
```

Đoạn mã 3.17. Test kiểm tra lỗi: ném ArgumentException và không gọi AddAsync

5.5 Test 3 — lấy CLB theo Id (bài tập)

```
[Fact]
public async Task GetClubById_WhenIdExists_ReturnsCorrectClub()
{
    var club = Club.Create("Tech Club", "Câu lạc bộ công nghệ");
    var mockRepo = new Mock<IClubRepository>();
    mockRepo.Setup(r => r.GetByIdAsync(club.Id, default)).ReturnsAsync(club);
    var handler = new GetClubByIdQueryHandler(mockRepo.Object);

    var result = await handler.HandleAsync(new GetClubByIdQuery(club.Id));

    Assert.NotNull(result);
    Assert.Equal(club.Id, result!.Id);
}
```

Đoạn mã 3.18. Dùng Setup để mock trả về CLB đã chuẩn bị

Đây là test bạn tự viết trong bài tập Ch03. Gợi ý thêm: viết test cho trường hợp Id không tồn tại, kết quả phải là null.

Chạy test và đặt tên test

```
$ dotnet test
Passed!  - Failed: 0, Passed: 5, Skipped: 0, Total: 5, Duration: 438 ms
```

- Tên test: **Phương thức_Tình huống_Kết quả mong đợi**, ví dụ CreateClub_WithEmptyName_ThrowsException
- Mỗi test kiểm tra đúng một hành vi, tên đọc lên là hiểu
- Test đỏ thì sửa code cài đặt, không sửa test cho xanh

Đoạn mã 3.19. Kết quả chạy test của bản đáp án

Bài tập Ch03 — EduHub Skeleton (HĐ1.3)

PHẦN VIỆC	ĐIỂM
Kiến trúc 4 lớp, đúng chiều tham chiếu	10
Infrastructure: InMemoryClubRepository (TODO 1)	15
Application: 4 handler, kiểm tra đầu vào (TODO 2 đến 5)	20
DI, ClubsController, Swagger (TODO 6 đến 8)	30
Unit test bằng Moq: 4 test có sẵn và 1 test tự viết	15
Pull Request và Conventional Commits	10

Bảng 3.11. Cơ cấu điểm bài tập chương 3 (tối đa 100, cộng thêm 10 nếu làm MembersController)

Starter: 02_Projects/ch03/starter. Mô tả đầy đủ trong de-bai.html và rubric.html cùng thư mục cha.

Nộp bài bằng Pull Request

```
git checkout -b feature/ch03-skeleton
# ... làm bài, chạy dotnet build và dotnet test ...
git add .
git commit -m "feat(infra): implement InMemoryClubRepository"
git commit -m "feat(api): add ClubsController with Swagger"
git commit -m "test(app): add GetClubById test with Moq"
git push -u origin feature/ch03-skeleton
# Mở Pull Request về main, mô tả what / why / how to test
```

Đoạn mã 3.20. Quy trình nộp bài chương 3

Mỗi thay đổi một commit có ý nghĩa, viết theo dạng type(scope): subject. Mô tả Pull Request nêu đã làm gì (what), vì sao (why), cách kiểm tra (how to test) và kèm ảnh **dotnet test**, Swagger UI.

Tổng kết chương 3

- **4 lớp**, phụ thuộc chỉ hướng vào lõi Domain
- **Interface và DI container**: lớp trong chỉ khai báo hợp đồng, Program.cs chọn bản cài đặt
- **Controller mỏng**: nhận request, gọi handler, chọn status code; route theo tên tài nguyên
- **Swagger và status code**: 200, 201, 204, 400, 404 cho từng tình huống
- **Test Application bằng Moq**: không cần database, kiểm tra kết quả và cách gọi kho

CLO	TRONG CHƯƠNG NÀY BẠN ĐÃ
CLO1	Dựng khung 4 lớp, giữ đúng Dependency Rule, dùng interface và DI
CLO2	Viết REST API với ASP.NET Core, có Swagger và status code chuẩn; nộp bài bằng Pull Request
CLO4	Dựng bộ khung EduHub để các chương sau mở rộng
CLO5	Thuyết trình nhóm, nhận phản biện và chỉnh sửa theo góp ý
CLO7	Tự nghiên cứu tài liệu chính thức để chuẩn bị bài trình bày

Bảng 3.12. Đối chiếu nội dung chương 3 với chuẩn đầu ra học phần

Câu hỏi tự kiểm tra

✔ CHECKPOINT – CHƯƠNG 3

1. Vì sao Domain không được tham chiếu Infrastructure? Điều gì xảy ra nếu vi phạm?
2. Handler nhận IClubRepository qua constructor. Ai quyết định bản cài đặt nào được đưa vào?
3. GET /api/clubs/{id} khi không có CLB nên trả status code nào? Còn POST khi tạo thành công?
4. Controller-based API khác Minimal API ở điểm nào? Vì sao EduHub chọn Controller?
5. Trong test, Setup và Verify của Moq khác nhau thế nào?

Xem trước Chương 4 — Xây dựng ứng dụng với .NET 10

🗄️ **EF Core và PostgreSQL**: migration, seeding, query

⚡ **Dapper**: truy vấn báo cáo cần raw SQL

📦 **Repository và Unit of Work**: khi nào cần, khi nào thừa; async toàn chuỗi

🔁 **DTO, mapping, validation, phân trang**

🚨 **ProblemDetails (RFC 9457) và Serilog**: lỗi chuẩn, log có cấu trúc

Chuẩn bị cho Chương 4

📖 Đọc tài liệu [1], [3], [4] ở mục tài liệu học tập của đề cương

🛠️ Hoàn thiện bài tập Ch03 và nộp Pull Request

🗣️ Nhóm thuyết trình Chương 4: nộp slide qua Pull Request trước 23:59 của ngày cách buổi trình bày 4 ngày

🐘 Kiểm tra PostgreSQL đã cài và kết nối được từ máy của bạn